



УНИВЕРЗИТЕТ У НОВОМ САДУ ФАКУЛТЕТ ТЕХНИЧКИХ НАУКА



УНИВЕРЗИТЕТ У НОВОМ САДУ
ФАКУЛТЕТ ТЕХНИЧКИХ НАУКА
НОВИ САД
Департман за рачунарство и аутоматику
Одсек за рачунарску технику и рачунарске комуникације

ИСПИТНИ РАД

Кандидат: Владимир Чајка
Број индекса: SV55/2024

Предмет: Паралелно програмирање
Тема рада: Паралелизација генетског алгоритма за решавање проблема
путујућег трговца

Ментор рада: др Душан Кењић

Нови Сад, јун, 2026

САДРЖАЈ

1.	Увод.....	4
1.1	Проблем путујућег трговца	4
1.2	Генетски алгоритам за TSP	5
1.3	Циљ пројекта	5
2.	Концепт решења.....	6
2.1	Учитавање градова и матрица удаљености	6
2.2	Репрезентација јединке и популације	6
2.3	Генетски оператори.....	6
2.3.1	Селекција родитеља	6
2.3.2	SCX укрштање	7
2.3.3	Инверзиона мутација	7
2.3.4	Елитизам.....	7
2.4	Серијска верзија алгоритма.....	7
3.	Опис решења	8
3.1	Модул главног програма (main.cpp)	8
3.2	Модул за аргументе командне линије (CommandLineOptions).....	8
3.3	Модул TSP инстанце (TspInstance).....	9
3.4	Структура јединке (Individual)	9
3.5	Модул генетског алгоритма (GeneticAlgorithm).....	10
3.6	Генерисање почетне популације.....	11
3.7	Евалуација популације.....	11
3.8	Селекција, укрштање и мутација.....	11
3.9	Сортирање и елитизам	11
3.10	Рачунање статистике	12
3.11	Island model.....	12
3.12	Мерење времена (ExecutionResult)	12
3.13	Упис резултата (ResultWriter).....	12
4.	Тестирање	15
4.1	Циљ тестирања	15
4.2	Тестирање учитавања података	15
4.3	Тестирање валидности рута	15
4.4	Тестирање серијске верзије.....	16

4.5	Тестирање паралелне верзије	16
4.6	Тестирање броја нити	16
4.7	Тестирање величине популације	17
4.8	Тестирање island модела	19
5.	Закључак	21

1. Увод

1.1 Проблем путујућег трговца

Проблем путујућег трговца, односно TSP (*Traveling Salesman Problem*), представља један од познатих комбинаторних оптимизационих проблема. Задатак је пронаћи најкраћу затворену туру која обилази сваки град тачно једном и враћа се у почетни град.

У овом пројекту решава се TSP инстанца са 52 града. Сваки град је задат својим идентификатором и координатама у равни, а растојање између два града рачуна се применом Еуклидске метрике. Циљ је да се пронађе што краћа рута, односно пермутација градова која минимизује укупну дужину затворене туре.

Овај проблем је погодан за примену еволутивних алгоритама јер број могућих тура расте факторијелно са бројем градова. Због тога је исцрпна претрага непрактична, па се користи приближни оптимизациони приступ заснован на генетском алгоритму.

1.2 Генетски алгоритам за TSP

Генетски алгоритам је метахеуристички оптимизациони алгоритам инспирисан природном еволуцијом. Основна идеја је да се кроз више генерација популација решења постепено побољшава применом селекције, укрштања, мутације и елитизма.

У овом пројекту једна јединка представља једну могућу TSP туру, односно пермутацију индекса градова. Квалитет јединке одређује се дужином затворене туре. Што је дужина туре мања, јединка је боља.

У свакој генерацији алгоритам сортира популацију по квалитету, задржава најбоље јединке елитизмом, бира родитеље, генерише потомке применом SCX укрштања и примењује инверзиону мутацију. Овај поступак се понавља до задатог броја генерација или до испуњења услова раног заустављања.

1.3 Циљ пројекта

Циљ пројекта је имплементација серијске и паралелне верзије генетског алгоритма за решавање проблема путујућег трговца у програмском језику C++. Паралелизација је реализована коришћењем *Intel TBB* библиотеке.

У оквиру пројекта имплементирани су следећи делови:

- серијска C++ верзија генетског алгоритма,
- паралелна C++/TBB верзија алгоритма,
- паралелна изградња матрице удаљености,
- паралелно генерисање почетне популације,
- паралелна евалуација популације,
- паралелно генерисање потомака,
- паралелно сортирање популације,
- паралелно рачунање статистике,
- контрола броја *TBB* нити,
- benchmark режим за мерење времена и убрзања,
- island model као напредна паралелна варијанта генетског алгоритма.

2. Концепт решења

2.1 Учитавање градова и матрица удаљености

Улазни подаци се читавају из текстуалне датотеке која садржи идентификатор града и његове координате. Сваки ред датотеке представља један град у формату:

$$id \times y$$

Након читавања градова, формира се матрица удаљености. Матрица садржи растојања између сваког пара градова, тако да се током рада генетског алгоритма растојање између два града може добити у константном времену.

Растојање између два града рачуна се Еуклидском формулом:

$$d(i, j) = \sqrt{(x_i - x_j)^2 + (y_i - y_j)^2}$$

Изградња матрице удаљености је погодна за паралелизацију, јер се сваки ред матрице може израчунати независно од осталих редова.

2.2 Репрезентација јединке и популације

Јединка у генетском алгоритму представља једну могућу TSP туру. У имплементацији је јединка моделована структуром *Individual*, која садржи:

- `route` – вектор индекса градова,
- `length` – укупну дужину затворене туре.

Рута мора бити валидна пермутација свих градова. То значи да сваки град мора да се појави тачно једном. На пример, ако постоји 52 града, рута мора имати 52 различита индекса из опсега од 0 до 51.

Популација је вектор јединки. У свакој генерацији популација се евалуира, сортира по дужини руте и на основу ње се формира нова популација.

2.3 Генетски оператори

2.3.1 Селекција родитеља

Селекција се врши на основу ранга јединки у сортираној популацији. Боље јединке имају већу вероватноћу избора, али и слабије јединке могу бити изабране, што доприноси разноврсности популације.

2.3.2 SCX укрштање

SCX (*Sequential Constructive Crossover*) конструише потомка корак по корак. Полази се од почетног града, а затим се на основу редоследа у родитељима и удаљености између градова бира следећи град. Овај оператор је погодан за TSP јер чува пермутациону природу решења.

2.3.3 Инверзиона мутација

Инверзиона мутација бира два индекса у рути и обрће редослед градова између њих. Ова мутација чува валидност пермутације, јер не додаје и не уклања градове, већ само мења редослед постојећих.

2.3.4 Елитизам

Елитизам омогућава да се најбоље јединке из тренутне генерације директно пренесу у следећу генерацију. На тај начин се спречава губитак најбољег до сада пронађеног решења.

2.4 Серијска верзија алгоритма

Серијска верзија алгоритма извршава све кораке један за другим. Прво се генерише почетна популација, затим се свака јединка евалуира рачунањем дужине руте. Након тога се популација сортира и почиње итеративни процес еволуције кроз генерације.

У свакој генерацији серијска верзија формира нову популацију тако што најбоље јединке преноси елитизмом, а преостале јединке генерише применом селекције, укрштања и мутације. Након формирања нове популације, све јединке се поново евалуирају и сортирају

3. Опис решења

Решење је организовано у више модула, при чему сваки модул има јасно дефинисану улогу. Основна идеја је да се раздвоје читавање TSP инстанце, конфигурација програма, логика генетског алгоритма, мерење времена и упис резултата. На овај начин је програм прегледнији, лакши за тестирање и погоднији за проширење.

Главни део решења чини класа `GeneticAlgorithm`, која садржи имплементацију серијске верзије, паралелне верзије и `island` модела. Паралелизација је реализована помоћу `Intel oneTBB` библиотеке, док се број нити може контролисати преко аргумента командне линије.

3.1 Модул главног програма (`main.cpp`)

Главни програм представља улазну тачку апликације.

```
int main(int argc, char** argv);
```

Функција `main` најпре обрађује аргументе командне линије и формира објекат `CommandLineOptions`. Након тога, ако је корисник задао број нити, поставља се ограничење TBB паралелизма помоћу `tbb::global_control`.

У зависности од режима покретања, програм може да изврши:

- серијску верзију алгоритма,
- паралелну верзију алгоритма,
- `benchmark` режим у коме се покрећу и серијска и паралелна верзија,
- `island model` ако је број острва већи од 1.

У стандардном режиму програм исписује најбољу пронађену туру, њену дужину и време извршавања. У `benchmark` режиму уписују се подаци потребни за анализу перформанси: серијско време, паралелно време, убрзање и ефикасност.

3.2 Модул за аргументе командне линије (`CommandLineOptions`)

Модул `CommandLineOptions` задужен је за обраду параметара које корисник прослеђује приликом покретања програма.

```
CommandLineOptions parseArguments(int argc, char** argv);
```

Подржани су параметри за подешавање величине популације, броја генерација, вероватноће укрштања, вероватноће мутације, удела елитних јединки, `seed` вредности, броја нити, као и параметара `island` модела.

Најважнији параметри су:

- `--pop` - величина популације,
- `--gen` - број генерација,


```

--pc - вероватноћа укрштања,
--pm - вероватноћа мутације,
--elite - удео елитних јединки,
--patience - број генерација без побољшања пре раног заустављања,
--seed - seed за генератор случајних бројева,
--parallel - покретање паралелне верзије,
--benchmark - покретање benchmark режима,
--threads - максималан број TBB нити,
--islands - број острва у island моделу,
--migration - интервал миграције,
--migrants - број миграната по острву.

```

Ако је аргумент непознат или недостаје његова вредност, програм пријављује грешку преко изузетка.

3.3 Модул TSP инстанце (`TspInstance`)

Класа `TspInstance` задужена је за учитавање градова и формирање матрице удаљености.

```
TspInstance(const std::string& filePath, bool useParallel = false);
```

Улазна датотека садржи ID града и његове координате. Након учитавања, формира се матрица удаљености између свих парова градова. Матрица је смештена као једнодимензиони вектор, што омогућава брз приступ растојању:

```
distance(fromIndex, toIndex)
```

Серијска верзија матрицу гради класичним угнежђеним петљама. Паралелна верзија користи `tbb::parallel_for`, јер се сваки ред матрице може израчунати независно од осталих редова.

Овај модул садржи и методе:

```

int cityCount() const;
const std::vector<City>& getCities() const;
double distance(int fromIndex, int toIndex) const;

```

Метода `cityCount` враћа број градова, `getCities` омогућава приступ учитаним градовима, а `distance` враћа унапред израчунато растојање између два града.

3.4 Структура јединке (`Individual`)

Јединка је представљена структуром `Individual`.

```
struct Individual {
    std::vector<int> route;
    double length;
};
```

Поље `route` представља редослед обиласка градова, односно једну TSP туру. Поље `length` представља укупну дужину те затворене туре.

Валидна рута мора да садржи сваки град тачно једном. Због тога је рута у суштини пермутација индекса градова. У Debug конфигурацији постоји додатна провера валидности руте, чиме се лакше откривају грешке у укрштању или мутацији.

3.5 Модул генетског алгоритма (`GeneticAlgorithm`)

Класа `GeneticAlgorithm` представља централни део решења. Она садржи комплетну логику серијског алгоритма, паралелне верзије и `island` модела.

Основне јавне методе су:

```
GAResult runSerial();
GAResult runParallel();
```

Метода `runSerial` покреће серијску верзију алгоритма. Метода `runParallel` покреће паралелну верзију, а ако је број острва већи од 1, покреће `island model`.

У оквиру алгоритма извршавају се следећи кораци:

1. генерисање почетне популације,
2. евалуација популације,
3. сортирање по дужини туре,
4. пренос елитних јединки,
5. селекција родитеља,
6. SCX укрштање,
7. инверзиона мутација,
8. формирање нове популације,
9. рачунање статистике,
10. провера услова за заустављање.

За серијску и паралелну верзију постоје одвојене помоћне функције. На пример, почетна популација може да се генерише серијски или паралелно:

```
createInitialPopulationSerial();
createInitialPopulationParallel();
```

Исто важи и за формирање нове популације, евалуацију, сортирање и рачунање просечне дужине.

3.6 Генерисање почетне популације

Почетна популација се састоји од насумично генерисаних валидних рута. Свака рута настаје тако што се формира вектор свих индекса градова, а затим се насумично измеша.

Серијска верзија генерише јединке једну по једну. Паралелна верзија користи `tbb::parallel_for`, јер се свака јединка може генерисати независно.

Да би резултати били стабилни и да не би дошло до конкурентног приступа једном генератору случајних бројева, свака јединка добија свој локални генератор. Seed се формира на основу основне seed вредности, индекса јединке и индекса острва.

3.7 Евалуација популације

Евалуација подразумева рачунање дужине сваке туре. Дужина туре добија се сабирањем растојања између суседних градова у рути, као и растојања од последњег града назад до првог.

```
double calculateRouteLength(const std::vector<int>& route) const;
```

Овај део је погодан за паралелизацију јер се дужина сваке јединке рачуна независно. Због тога паралелна верзија користи `tbb::parallel_for`.

3.8 Селекција, укрштање и мутација

Селекција родитеља реализована је на основу ранга јединки у сортираној популацији. Боље јединке имају већу вероватноћу избора, али и слабије јединке могу бити изабране. Ово помаже очувању разноврсности популације.

Укрштање је реализовано SCX оператором. Овај оператор гради потомка тако што у сваком кораку бира следећи град на основу два родитеља и удаљености до кандидата. На тај начин се користи информација о структури TSP проблема.

Мутација је реализована као инверзиона мутација. Бирају се два индекса у рути и обрће се редослед градова између њих. Ова операција не нарушава валидност руте јер не додаје и не уклања градове.

3.9 Сортирање и елитизам

Након евалуације, популација се сортира по дужини туре. Најбоље јединке налазе се на почетку вектора.

Серијска верзија користи `std::sort`, док паралелна верзија користи:

```
tbb::parallel_sort
```

Елитизам омогућава да се одређени број најбољих јединки директно пренесе у следећу генерацију. Ово спречава губитак најбољег до сада пронађеног решења.

3.10 Рачунање статистике

За анализу рада алгоритма чува се историја по генерацијама. За сваку генерацију бележе се:

- редни број генерације,
- најбоља дужина туре,
- просечна дужина популације.

Просечна дужина у серијској верзији рачуна се обичном петљом, док паралелна верзија користи `tbb::parallel_reduce`. Овај облик паралелизације омогућава да свака нит израчуна локалну суму, након чега се локалне суме спајају у коначан резултат.

3.11 Island model

Island model је имплементиран као посебна варијанта паралелног алгоритма. Уместо једне популације, користи се више популација, односно острва.

Свако острво има своју популацију и еволуира независно од осталих острва. Након одређеног броја генерација врши се миграција. Најбоље јединке са једног острва пребацују се на следеће острво у прстенастој топологији.

Миграција се врши тако што најбоље јединке са изворног острва замењују најлошије јединке на циљном острву. Након миграције, популација се поново сортира.

У коначној верзији island model користи два нивоа паралелизма. Први ниво је паралелизам између острва, док други ниво представља паралелизација унутар сваког острва: генерисање потомака, евалуација, сортирање и рачунање статистике.

3.12 Мерење времена (`ExecutionResult`)

Модул `ExecutionResult` служи за мерење времена извршавања алгоритма.

```
ExecutionResult runSolver(GeneticAlgorithm& algorithm, bool parallel);
```

У стандардном режиму мери се време извршавања самог алгоритма. У benchmark режиму користи се посебна функција која у мерење укључује и припрему TSP инстанце, односно учитавање података и изградњу матрице удаљености. На тај начин benchmark реалније пореди серијску и паралелну верзију.

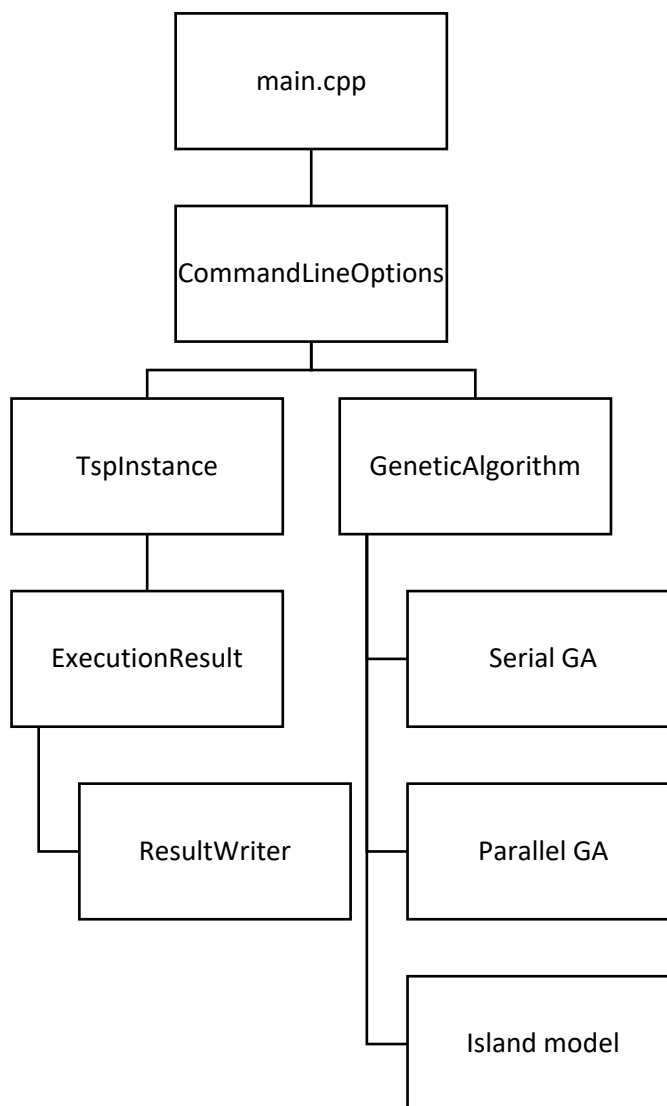
3.13 Упис резултата (`ResultWriter`)

Модул `ResultWriter` задужен је за приказ и упис резултата. Најбоља рута се конвертује из индекса градова у ID-јеве градова, након чега се може ротирати тако да почиње од задатог ID-ја.

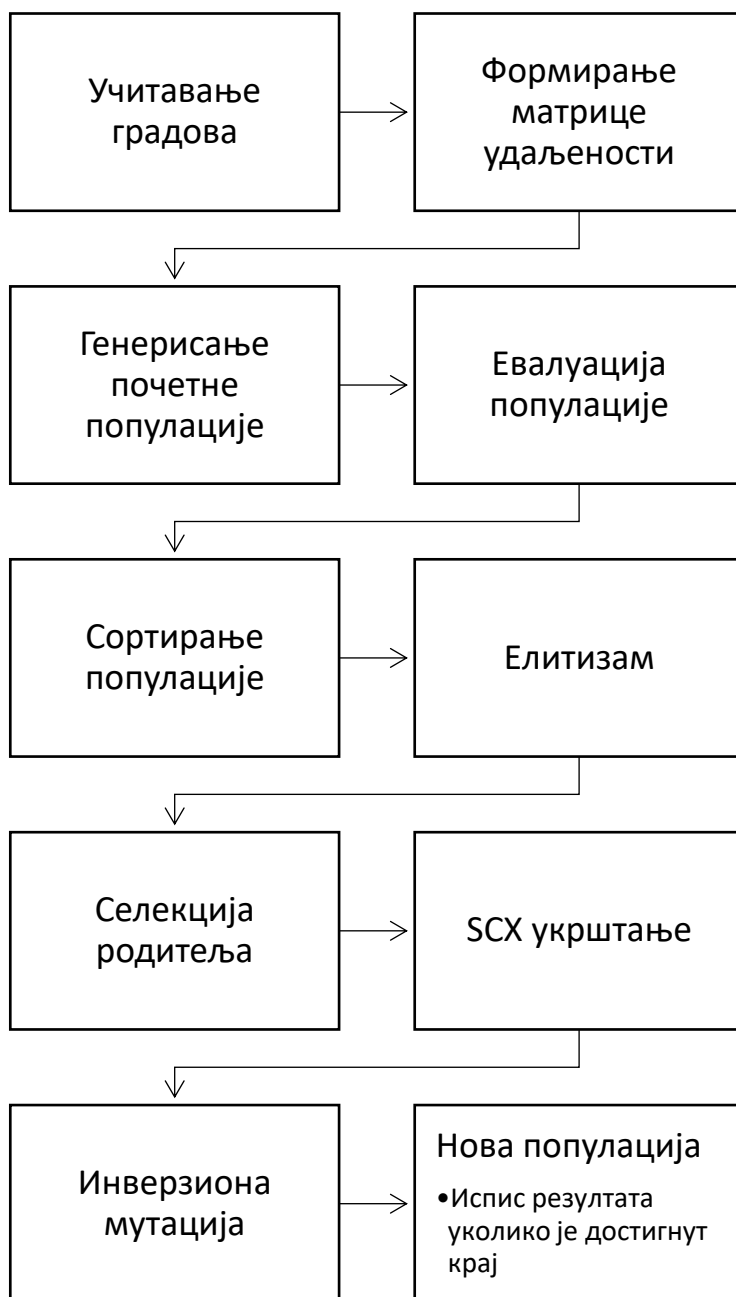
Програм генерише следеће излазне датотеке:

`best_route.txt` - најбоља пронађена тура, њена дужина и време извршавања,
`history.csv` - историја конвергенције по генерацијама,
`benchmark.csv` - резултати `benchmark` тестова, укључујући време, убрзање и ефикасност.

У `benchmark` датотеци се за свако покретање чувају параметри експеримента, број нити, број острва, број генерација, најбоља дужина, време извршавања, убрзање и ефикасност.



Слика 1: Архитектура програма



Слика 2: Ток генетског алгоритма

4. Тестирање

4.1 Циљ тестирања

Циљ тестирања је да се провери исправност имплементације генетског алгоритма, исправност серијске и паралелне верзије, као и понашање програма у benchmark режиму. Поред функционалне исправности, тестирање обухвата и мерење перформанси, односно анализу времена извршавања, убрзања и ефикасности паралелизације.

Тестирање је подељено на неколико целина:

- провера учитавања TSP инстанце,
- провера валидности генерисаних рута,
- провера серијске верзије алгоритма,
- провера паралелне верзије алгоритма,
- провера benchmark режима,
- провера island модела,
- анализа резултата за различит број нити и различите величине популације.

4.2 Тестирање учитавања података

Први корак тестирања односи се на учитавање улазне датотеке data_tsp.txt. Датотека садржи ID града и његове координате. Програм треба да успешно прочита све градове и формира матрицу удаљености.

Очекивано понашање је да програм при исправној датотеци прочита 52 града и настави извршавање без грешке. Уколико датотека не постоји или није доступна, програм треба да пријави грешку и прекине извршавање.

4.3 Тестирање валидности рута

Пошто једна јединка представља TSP туру, свака генерисана рута мора бити валидна пермутација градова. То значи да рута мора да садржи сваки град тачно једном, без понављања и без изостављених градова.

У Debug конфигурацији постоји додатна провера валидности рута. Ова провера је корисна током развоја јер може да открије грешке у генерисању почетне популације, укрштању или мутацији.

Код финалних резултата проверено је да најбоља пронађена рута садржи свих 52 различита ID-ја града и да се затворена тура враћа у почетни град.

4.4 Тестирање серијске верзије

Серијска верзија алгоритма тестирана је као основна, референтна имплементација. Она извршава све кораке алгоритма један за другим: генерисање популације, евалуацију, сортирање, селекцију, укрштање, мутацију и формирање нове генерације.

Очекивано је да програм успешно пронађе туру, испише резултат и упише датотеке `best_route.txt` и `history.csv`.

4.5 Тестирање паралелне верзије

Паралелна верзија тестирана је са истим параметрима као серијска верзија, уз додатни аргумент `--parallel`. Циљ је да се провери да паралелна верзија даје валидно решење и да се понаша стабилно за исти `seed`.

Паралелна верзија користи ТВВ за паралелно генерисање популације, евалуацију, генерисање потомака, сортирање и рачунање статистике. Тестирањем је потврђено да се програм извршава без грешке и да генерише валидну туру.

4.6 Тестирање benchmark режима

Benchmark режим служи за поређење серијске и паралелне верзије. У овом режиму програм покреће обе верзије, мери њихово време извршавања и рачуна убрзање и ефикасност.

Резултати benchmark тестова уписују се у датотеку `benchmark.csv`. За свако покретање бележе се параметри експеримента, број нити, број острва, најбоља дужина туре, време извршавања, `speedup` и `efficiency`.

4.7 Тестирање броја нити

Један од главних експеримената односи се на утицај броја нити на време извршавања. Величина популације и број генерација су фиксирани, док се број нити мења.

Коришћени бројеви нити су:

1, 2, 4, 8, 12, 16

Циљ овог теста је да се покаже како се паралелна верзија скалира са повећањем броја нити. Очекује се да паралелна верзија са једном нити може бити спорија од серијске због overhead-а ТВВ задатака, док се са већим бројем нити добија убрзање.

Број нити	Серијско време [s]	Паралелно време [s]	Убрзање	Ефикасност
1	10.140160	12.615956	0.803757x	0.803757
2	10.171175	6.626194	1.534995x	0.767498
4	10.399327	3.482227	2.986402x	0.746600
8	10.183487	2.328604	4.373216x	0.546652
12	10.182500	2.000008	5.091230x	0.424269
16	10.162817	1.974984	5.145773x	0.321611

Табела 1: Утицај броја нити на перформансе

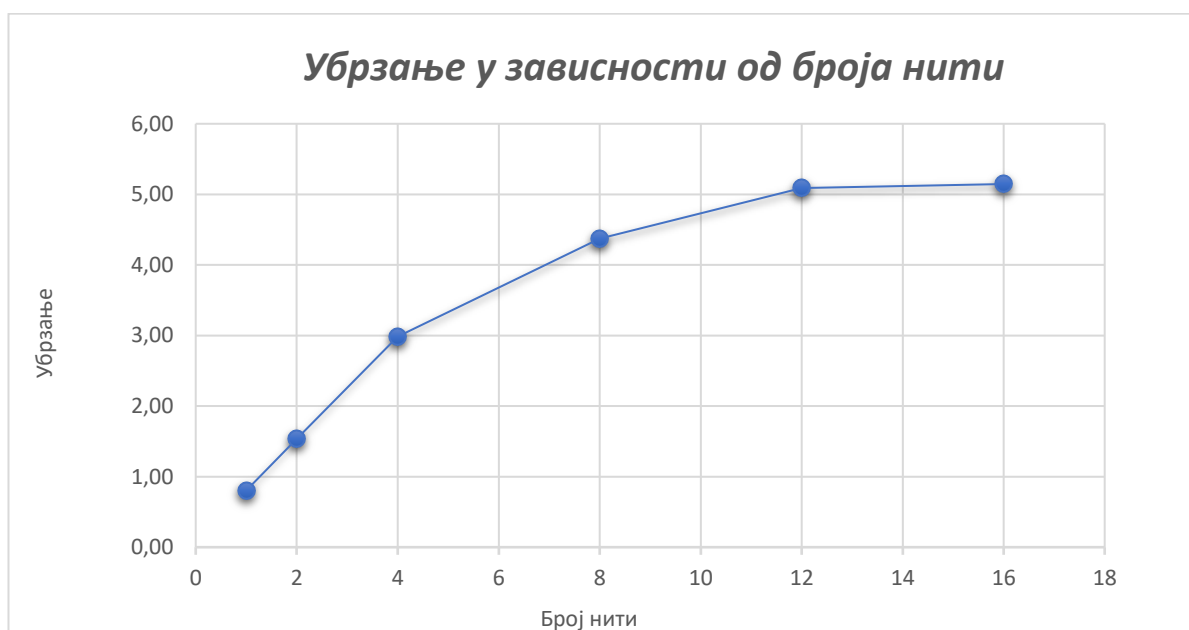


График 1: Убрзање у зависности од броја нити

4.8 Тестирање величине популације

Други експеримент испитује утицај величине популације на исплативост паралелизације. Број нити и број генерација су фиксирани, док се величина популације мења.

Коришћене величине популације су:

1000, 5000, 10000, 20000

Циљ овог теста је да се провери како се алгоритам понаша када се количина посла повећава. Како величина популације расте, расте и време извршавања и серијске и паралелне верзије. Међутим, паралелна верзија у свим случајевима остаје знатно бржа од серијске.

Из резултата се види да је убрзање стабилно и креће се приближно између 4.14x и 4.55x. Највеће убрзање у овом експерименту добијено је за популацију од 1000 јединки, док се код већих популација убрзање благо смањује. То показује да паралелизација доноси значајну корист за све тестиране величине популације, али да у овом конкретном експерименту повећање популације није довело до већег убрзања.

Могуће објашњење је да са већим популацијама расте и количина посла који није савршено паралелан, као и трошак сортирања, распоређивања задатака и приступа меморији. Због тога се добија стабилно, али не и растуће убрзање.

Популација	Серијско време [s]	Паралелно време [s]	Убрзање	Ефикасност
1000	2.009836	0.441431	4.553004x	0.569126
5000	10.14560	2.313494	4.385401x	0.548175
10000	20.314499	4.703649	4.318881x	0.539860
20000	40.906489	9.878115	4.141123x	0.517640

Табела 2: Утицај величине популације

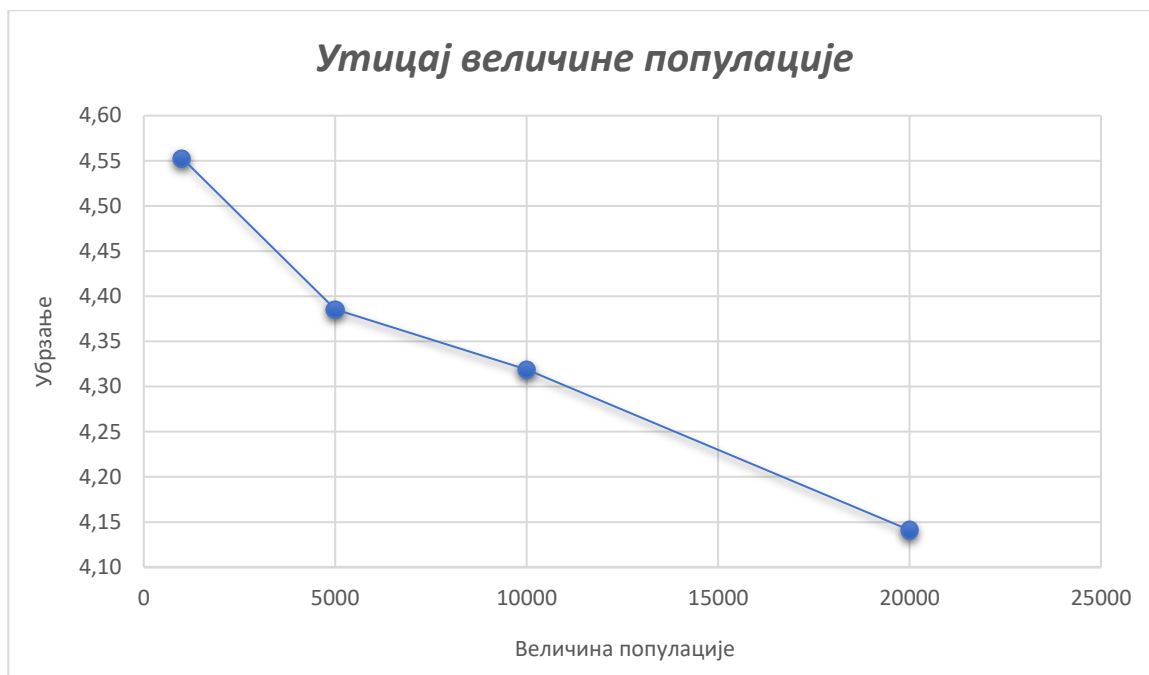
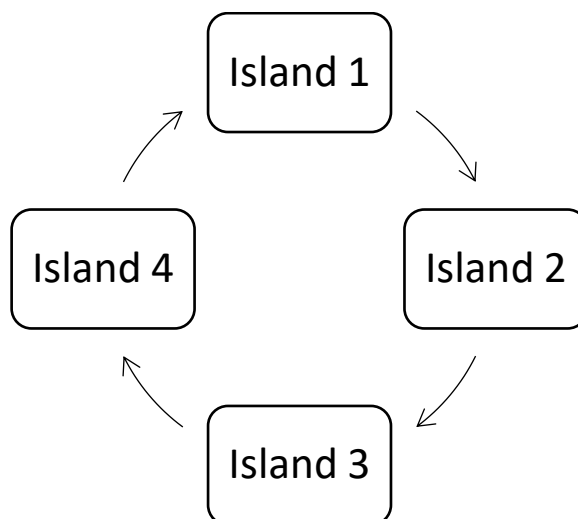


График 2: Утицај величине популације

4.9 Тестирање island модела



Слика 3: Исланд модел

Island model је тестиран тако што је укупна величина популације задржана иста, док је популација подељена на различит број острва.

Коришћене конфигурације су:

2 острва x 8000 јединки

4 острва x 4000 јединки

8 острва x 2000 јединки

Циљ овог теста је да се испита да ли подела популације на више независних острва утиче на време извршавања и квалитет пронађеног решења. Острва еволуирају независно између миграција, док миграција представља тачку синхронизације.

Број острва	Популација по острву	Укупна популација	Серијско време [s]	Време исланд модела [s]	Убрзање	Ефикасност
2	8000	16000	33.028739	7.162250	4.61150x	0.576438
4	4000	16000	32.838073	6.808286	4.82325x	0.602906
8	2000	16000	32.700857	6.661368	4.90903x	0.613629

Табела 3: Резултати исланд модела

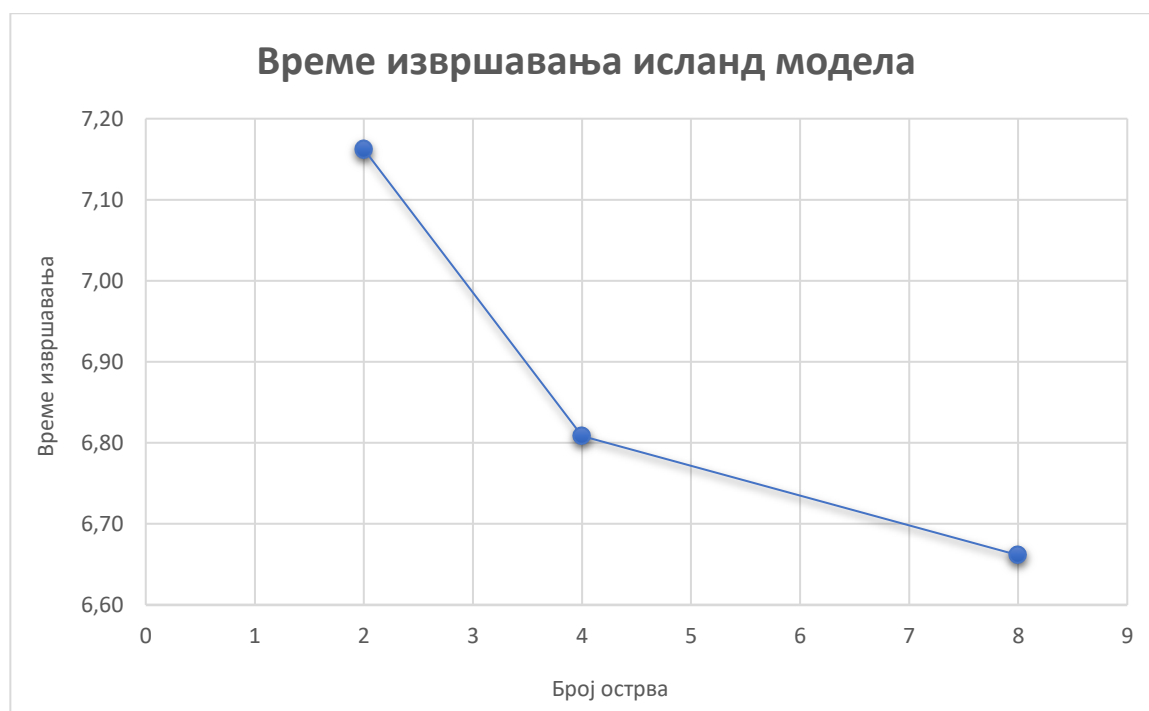


График 3: Време извршавања исланд модела

5. Закључак

У оквиру пројекта имплементиран је генетски алгоритам за решавање проблема путујућег трговца, као и његова паралелна верзија помоћу Intel oneTBB библиотеке. Решење је организовано модуларно, тако да су раздвојени читавање података, формирање матрице удаљености, логика алгоритма, мерење времена и упис резултата.

Паралелизација је примењена на делове алгоритма који се могу независно извршавати: изградњу матрице удаљености, генерисање популације, евалуацију, формирање нове популације, сортирање и рачунање статистике. Поред класичне паралелне верзије, имплементиран је и island model, где више независних популација еволуира паралелно уз периодичну миграцију најбољих јединки.

Резултати показују да паралелна верзија у свим тестираним конфигурацијама са више нити постиже значајно краће време извршавања у односу на серијску верзију. Највећи измерени speedup износи приближно 5.15x. У експерименту са различитим величинама популације убрзање је остало стабилно, али није расло са повећањем популације. Island model је показао благо побољшање времена извршавања са повећањем броја острва. Најбоља пронађена дужина туре износи 7544.365902, при чему је добијена валидна затворена тура са свих 52 града.

На основу резултата може се закључити да решење испуњава захтеве пројектног задатка и показује да се генетски алгоритам за TSP може успешно паралелизовати коришћењем Intel oneTBB библиотеке. Паралелизација је дала најбоље резултате у експериментима са већим бројем нити, док је код различитих величина популације показала стабилно убрзање у односу на серијско извршавање.